

1. Given this table definition:

```
CREATE TABLE MyTable (ID numeric(10,5) PRIMARY KEY, Comments VARCHAR(200));  
ALTER TABLE MyTable ADD CONSTRAINT UniqueID UNIQUE CLUSTERED (ID)
```

A **PRIMARY KEY** is by default backed by a clustered index. There can be only one clustered index per table.



2. Given the following two table definitions:

```
CREATE TABLE Customer(CustName NVARCHAR(4000) COLLATE Latin1_General_CI_AI)
CREATE TABLE Supplier(SuppName NVARCHAR(4000) COLLATE SQL_Latin1_General_CP1_CI_AI)
```

This query raises an error:

```
SELECT * FROM Customer c
JOIN Supplier s
ON c.CustName = s.SuppName
```

How would you mitigate the following error messages? Msg 468, Level 16, State 9, Line 630

Cannot resolve the collation conflict between "SQL_Latin1_General_CP1_CI_AI" and "Latin1_General_CI_AI" in the equal to operation.

Add the clause `COLLATE Latin1_General_CI_AI` to the join predicate.



3. The following is a stored procedure stub that saves away an order:

```
CREATE PROCEDURE SaveOrder      -- parameters

BEGIN TRY

BEGIN TRANSACTION      -- code to save an order goes here

COMMIT TRANSACTION

END TRY

BEGIN CATCH      -- insert error details to an error log table in the same db

ROLLBACK

END CATCH
```

Which statement correctly identifies the flaw in the design?

The error log insert comes before the rollback, which results in the error log insert getting rolled back.



4. What is a database schema?

A schema is a named container for database objects



5. Given the following:

```
CREATE TABLE Sales (OrderId int, OrderItemId int, Quantity int, UnitPrice numeric(6,2));  
INSERT INTO Sales (OrderId, OrderItemId, Quantity, UnitPrice)  
VALUES  
    (1, 1, 20, 0.6),  
    (1, 2, 10, 1.50),  
    (2, 1, 15, 0.75),  
    (2, 2, 42, 43),  
    (3, 1, 5, .10);  
  
SELECT OrderId, SUM(Quantity) as TotalQuantity, SUM(Quantity * UnitPrice) as TotalPrice  
FROM Sales  
GROUP BY OrderId
```

The following result set is returned:

OrderId	TotalQuantity	TotalPrice
1	30	25.00
2	57	1817.25
3	5	0.50

What could you add to the query to eliminate Order Id 3 as well as any orders with a total price more than 1,000?

HAVING SUM(Quantity * UnitPrice) BETWEEN 10 AND 1000



6. In SQL Server 2016 and higher, what command would you use to drop a database that will not cause an error if the database has not been created?

```
DROP DATABASE IF EXISTS database_name
```



7. The following stored procedure stub deletes product(s) by category:

```
CREATE PROCEDURE DeleteProductByCategory
    -- parameters
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION
        -- code to delete product by category goes here
        IF @@ROWCOUNT > 0
            COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        ROLLBACK
    END CATCH
END
GO
```

When the stored procedure executes, it occasionally returns an error. Which correctly identifies the cause of the error?

The `DeleteProductByCategory` stored procedure always executes the `BEGIN TRANSACTION` statement, but it only executes the `COMMIT TRANSACTION` statement if one or more products are deleted.



8. You have an outermost transaction and one transaction nested inside it. Which transaction(s) is/are rolled back when you use a `ROLLBACK TRANSACTION` without any arguments?

Both the transactions are rolled back



9. How would you execute the following stored procedure to make use of the output parameter `lastSalesDate` ?

```
CREATE PROCEDURE ProductLastSalesDate
@productID INT,
@lastSalesDate DATETIME OUTPUT
AS
SET NOCOUNT ON;
SELECT @lastSalesDate = MAX(SalesDate)
FROM Sales
WHERE ID = @productID;
```

```
DECLARE @lastDate DATETIME;
EXECUTE ProductLastSalesDate @productID = 123, @lastSalesDate = @lastDate OUTPUT;
PRINT CONVERT (VARCHAR, @lastDate, 120);
```



10. Which **JOIN** keyword results in zero records from the right table when no matches are found?

LEFT JOIN



11. What is the difference between unique and non-unique indexes?

With a unique index, **INSERT** operations are rolled back for duplicate values, whereas a non-unique index allows the **INSERT** operations with no error. ✓

12. Which function can you use to extract the number of sales from a JSON string?

JSON_VALUE()



13. Given the following table, how would you create a clustered index on its column named TestCol1?

```
CREATE TABLE dbo.TestTable  
  (TestCol1 int NOT NULL,  
   TestCol2 nchar(10) NULL,  
   TestCol3 nvarchar(50) NULL);  
  
GO
```

```
CREATE CLUSTERED INDEX IX_TestTable_TestCol1 ON dbo.TestTable (TestCol1);
```



14. Given the following dataset and procedure, which code snippet would return a value of 1 from the OUTPUT parameter?

DataSet:

UserID	userName
-----	-----
1	User1
1	User2
2	User3
3	User4
4	User5
5	User6
6	User7
7	User8

Procedure:

```
CREATE PROCEDURE GetUserCount
    @userID INT = NULL,
    @userCount INT OUTPUT
AS
BEGIN
    IF @userID IS NULL
        RETURN(-1)
    SET @userCount = (SELECT COUNT(*) FROM Users WHERE UserID = @userID)
END
```

EXEC GetUserCount 7, @userCount=@result OUTPUT



15. Which data type would best store a bank balance up to 1 billion dollars with four digits of scale?

`decimal(14,4)`



16. How will you create the following table `Tower` in T-SQL and name the primary key `PK_tower` ?

name	number	floors	color
----	----	----	----
ron	54	12	brown
	Primary Key		Default to white

<- sample values
<- constraints

```
CREATE TABLE Tower(  
    name nvarchar(20),  
    number int,  
    floors int,  
    color nvarchar(20) DEFAULT('white'),  
    CONSTRAINT PK_tower PRIMARY KEY CLUSTERED (number)  
);
```



17. How many rows does the following code return to the console?

```
CREATE TABLE BAR(  
  item nvarchar(20),  
  arrival date  
);  
  
INSERT INTO BAR VALUES ('TU785', '2021/05/12');  
INSERT INTO BAR VALUES ('TU777', '2021/05/13');  
INSERT INTO BAR VALUES ('TU777', '2021/05/1');  
INSERT INTO BAR VALUES ('TU777', '2021/05/26');  
INSERT INTO BAR VALUES ('TU777', '2021/05/28');  
  
SELECT * FROM BAR  
WHERE arrival BETWEEN '2020/05/21' AND '2021/05/01';
```

18. Given the following:

```
CREATE TABLE Orders (OrderId Int, CustId int);
CREATE TABLE OrderItems (OrderItemId int, OrderId int, SKU VARCHAR(50), Qty int)
INSERT INTO Orders(OrderID, CustID) VALUES
    (1, 1001), (2, 2002), (3, 3003)
INSERT INTO OrderItems (OrderItemId, OrderId, Qty, SKU) VALUES
    (1, 1, 10, 'Part1'),
    (2, 1, 5, 'Part5'),
    (3, 2, 15, 'Part2'),
    (4, 2, 20, 'Part1')
```

The following query attempts to get the smallest order item quantity:

```
SELECT OrderID, QTY
FROM OrderItems
WHERE ROW_NUMBER() OVER (PARTITION BY OrderId ORDER BY Qty ASC) = 1
```

It produces the error message:

```
Msg 4108, Level 15, State 1, Line 214
Windowed functions can only appear in the SELECT or ORDER BY clauses.
```

How would you change the query to produce the desired result without any error?

```
SELECT OrderId, Qty
FROM (
    SELECT OrderID, Qty,
           ROW_NUMBER() OVER (PARTITION BY OrderId ORDER BY Qty ASC) as rn
    FROM OrderItems
) sub
WHERE rn = 1;
```

